Title: Studex (Student Management System)

Motivation: Managing student data manually can be difficult and likely to cause errors. This project aims to develop a user-friendly program that automates the process of managing student records, thereby increasing efficiency and reducing mistakes

Concept (Problem Statement): The goal is to build a simple command-line Student Management System using the C programming language. The system should allow users to store and manage information for up to 100 students, including adding, viewing, updating, and deleting student data.

Design: Students are required to follow the design constraints set out for the implementation of OEL:

Introduction and Requirements: This program is a console-based application developed in C, aimed at managing student data efficiently. It uses basic data structures and requires only a standard C compiler to run. The key features include adding new student records, displaying all stored records, updating existing data, and deleting records as needed.

Data Structure Selection: To hold student details, the program uses a struct that includes the student's ID, name, and marks. An array of these structures serves as the main storage, allowing quick access and simple management of up to 100 student entries.

Basic Implementation: The Student Management System is implemented in C using a structure to store student details like ID, name, and marks. An array holds all student records, with functions to add, display, update, and delete students. A menu-driven interface allows users to perform these operations easily through the console.

Performance Testing and Analysis: For the given limit of 100 students, the program performs efficiently. Most operations, such as adding or deleting records, work in linear time, which is suitable for small datasets. Displaying records iterates over the array and is also handled smoothly.

Optimization and Advanced Features: To enhance the system, dynamic data structures like linked lists could be used for flexible storage. Adding persistent storage via file handling would allow data saving and retrieval. Also, input validation and exception handling would make the system more reliable.

Extensions and Creativity: The system can be expanded by adding features like sorting and searching student records, implementing automatic grading based on marks, and including user authentication for security. Additionally, creating a graphical user interface would improve usability and make the system more interactive.

Deliverables

Background/Theory: Student management systems organize and store academic information efficiently. Using data structures like arrays and structs in C provides a foundation for handling records systematically. This approach helps maintain consistency and quick access to student data in educational institutions.

Procedure / Methodology: The program uses a menu-driven interface to perform CRUD operations on student data stored in an array. Functions handle adding, displaying, updating, and deleting student records interactively. This method simplifies user interaction and ensures organized data management through modular code design.

Data Collection (If required): Student data such as ID, name, and marks are entered manually through the program interface during runtime. No external data sources are needed for this implementation. Data is temporarily stored in memory for immediate processing and manipulation.

Analysis: Operations on the data are performed in real-time with linear search and updates. The system's performance is suitable for up to 100 records, with acceptable response times for all tasks. This ensures the program runs smoothly without noticeable delay during use.

Results: The program successfully manages student records, allowing users to add, view, update, and delete data as intended. It maintains data integrity within the fixed array limit. User input is handled correctly, and feedback messages confirm operation success or failure.

Discussion on Results: The system works well for small datasets, demonstrating the effectiveness of array-based storage and modular functions. However, scalability and persistence are limited without further enhancements. Incorporating file handling and dynamic memory would make the system more versatile.

Concluding Remarks: This project provides a simple and functional student management system. Future improvements can include dynamic storage, persistent data saving, and a more user-friendly interface. The system lays a strong foundation for building more complex educational software.

Reference:

- Course Materials: Programming Fundamentals Lectures & Lab Guides
- GeeksforGeeks – Data Structures and Algorithms in C
- W3Schools – File Handling and User Input in C

Flowchart / Block diagram: